Prompt Design Explanation

This documents the system prompt and user prompt used at each LLM step, explains the reasoning behind each prompt's design, and shows at least one example of a prompt that was revised after testing.

Step 1 — Query Planner

Call type	LLM (Groq — LLaMA 3.3 70B)
Input	state["topic"] — the raw user topic string
Output	state["queries"] — list of 4 search query strings

System Prompt:

```
You are a research strategist. Given a news topic, generate exactly 4 focused
web search queries that together cover the topic from four complementary angles:
  1. Recent news and current developments
  2. Background, history, and context
  3. Expert analysis or opinion
  4. Future outlook or implications

Respond ONLY with a valid JSON object — no explanation, no markdown:
{
  "queries": ["query 1", "query 2", "query 3", "query 4"],
  "angles":  ["recent news", "background", "expert opinion", "future implications"]
}
```

User Prompt:

```
Topic: "ipl 2025"
```

Why this prompt is shaped this way

The four numbered angles in the system prompt are not decorative — they are constraints that prevent the model from generating four near-identical variations of the same query. Without this structure, the model defaults to rephrasing the topic with minor synonymic variation, which produces redundant search results.

The "ONLY a valid JSON" instruction with no explanation and no markdown is essential because Step 2 calls parse_json_response() on the output. Any preamble or trailing text breaks the JSON parser. The schema is shown explicitly (not described) because the model is more reliable when it sees the exact format it should produce.

Prompt iteration — what changed

First version (did not work):

```
Generate 4 search queries for the topic. Return them as a JSON array.
```

This produced a bare array like ["query1", "query2", ...] with no angle labels, and occasionally returned a Python list syntax (single quotes) that failed JSON parsing. The revised version adds the explicit schema showing both "queries" and "angles" keys, forcing double-quoted JSON output and preserving the angle metadata for traceability.

Step 2 — Web Search (Tool Call — Serper API)

Call type	External Tool — Serper API (NOT an LLM call)
Input	state["queries"] — 4 search query strings from Step 1
Output	state["all_articles"] — flat deduplicated list of article dicts: {title, snippet, source, date}

This step makes no LLM call. It calls the Serper web search API once per query and collects real, dated article results. There is no system or user prompt here — instead, the relevant design decisions are in how the tool is called, how failures are handled, and how the output is structured before being passed to Step 3.

Tool call code (from tools.py):

```
def serper_search(query: str, num_results: int = 5) -> list[dict]:
    headers = { "X-API-KEY": api_key, "Content-Type": "application/json" }
    payload = { "q": query, "num": num_results }
    response = requests.post(SERPER_URL, headers=headers, json=payload)
    response.raise_for_status()
    results = []
    for item in response.json().get("organic", [])[:num_results]:
        results.append({
            "title"  : item.get("title",   ""),
            "snippet": item.get("snippet", ""),
            "source" : item.get("link",    ""),
            "date"   : item.get("date",    "N/A"),
        })
    return results
```

How Step 2 calls the tool (from steps.py):

```
for query in state["queries"]:
    try:
        results = serper_search(query, num_results=4)
        raw_results[query] = results
    except Exception as exc:
        print(f"Search failed for '{query}': {exc}")
        raw_results[query] = []   # chain continues with remaining queries
```

Why a tool call and not an LLM call

LLMs have a training data cutoff — they cannot retrieve events that happened after their training ended, and they frequently hallucinate plausible-sounding recent facts with high confidence. A tool call grounds the entire rest of the chain in real, dated web content. Every theme, summary, and briefing section produced in Steps 3–6 traces back to actual articles retrieved here.

Serper was chosen specifically because its free tier requires no credit card, returns a clean JSON response with an 'organic' array, and includes publication dates alongside snippets. The date field is important for news topics where recency matters — it lets Step 3 know whether an article is from last week or last year.

Key design decisions in this step

Each query is executed in a separate try/except block rather than wrapping the entire loop. This means a single failed search (network timeout, rate limit) does not abort the other three — the chain degrades gracefully. Results are deduplicated by URL before being written to state["all_articles"], preventing the same article from being summarised twice if it appeared in multiple query results.

The output schema — four fields per article: title, snippet, source, date — was kept minimal deliberately. Step 3 receives exactly what it needs to produce a structured summary and nothing

more. Passing the full Serper response object (which includes rank, position, and other metadata) would increase token usage in Step 3 without adding analytical value.

Step 3 — Article Summariser

Call type	LLM (Groq — LLaMA 3.3 70B)
Input	state["all_articles"] — list of article dicts from Serper
Output	state["summaries"] — list of normalised summary dicts

System Prompt:

```
You are a news analyst. For each article, extract a structured summary.

Respond ONLY with a valid JSON array — no explanation, no markdown:
[
  {
    "article_id"  : 1,
    "headline"    : "one-sentence summary of what the article says",
    "key_facts"   : ["concrete fact 1", "concrete fact 2", "concrete fact 3"],
    "sentiment"   : "positive | negative | neutral | mixed",
    "source_name" : "domain name only (e.g. bbc.com)"
  }
]
Include one object per article. Keep key_facts as specific claims, not opinions.
```

User Prompt:

```
Topic: "ipl 2025"

Articles:
[Article 1]
Title   : MI vs CSK: Mumbai Indians win by 6 wickets
Source  : https://cricbuzz.com/...
Date    : Apr 12, 2025
Snippet : Mumbai Indians chased down 172 with 4 balls to spare ...
...(remaining articles follow same format)...
```

Why this prompt is shaped this way

The one-object-per-article constraint and the fixed schema prevent the model from merging articles or skipping low-quality ones. The "key_facts as specific claims, not opinions" instruction was added after testing revealed the model frequently listed vague observations like "the match was exciting" instead of concrete facts like "Rohit Sharma scored 78 off 44 balls". The sentiment enum is deliberately narrow — four values only — to prevent the model from inventing values that break Step 4's aggregation.

The user prompt formats articles as a numbered block (not a JSON dump) because the model summarises more accurately when the input resembles a document it would analyse, rather than a data structure it must parse before summarising.

Prompt iteration — what changed

First version of the instruction for key_facts:

```
"key_facts": ["important points from the article"]
```

This produced 1–6 items per article of inconsistent specificity. Changing it to:

```
"key_facts": ["concrete fact 1", "concrete fact 2", "concrete fact 3"]
```

anchored the model to exactly three items and — critically — the word 'concrete' shifted outputs from opinions to verifiable claims, which Step 4 can reason about reliably.

Step 4 — Theme Extractor

Call type	LLM (Groq — LLaMA 3.3 70B)
Input	state["summaries"] — normalised summary dicts from Step 3
Output	state["themes"] — theme analysis dict with themes[], contradictions, consensus

System Prompt:

```
You are a senior intelligence analyst. Given structured article summaries,
identify the major cross-cutting themes in the coverage.

Respond ONLY with a valid JSON object — no explanation, no markdown:
{
  "themes": [
    {
      "name"               : "short theme label",
      "description"        : "2-3 sentence explanation of this theme",
      "supporting_articles": [1, 2],
      "significance"       : "why this matters beyond the immediate news"
    }
  ],
  "contradictions"     : ["description of conflicting claims between sources"],
  "consensus"          : "what all or most sources agree on (1-2 sentences)",
  "overall_sentiment"  : "positive | negative | neutral | mixed",
  "key_entities"       : ["most-mentioned people, organisations, or places"]
}
Identify 3-5 themes. Be specific — name facts, not vague generalities.
```

Why this prompt is shaped this way

The supporting_articles array is the key structural constraint in this prompt. By forcing the model to cite which article IDs support each theme, it cannot invent themes that have no basis in the retrieved data. This also makes the output auditable — during a demo, the evaluator can open state_*.json and verify that a theme's supporting_articles actually say what the model claims.

The "3-5 themes" instruction prevents both over-fragmentation (one theme per article) and over-consolidation (everything collapsed into one meta-theme). The "Be specific — name facts, not vague generalities" instruction was the single most impactful addition during testing. Without it, theme descriptions defaulted to generic summaries like "the IPL continues to attract interest" rather than specific observations like "three teams have won matches by fewer than 5 runs, suggesting unusually competitive matches this season".

Step 5 — Briefing Writer

Call type	LLM (Groq — LLaMA 3.3 70B)
Input	state["themes"] from Step 4, state["summaries"] from Step 3
Output	state["draft_briefing"] — structured Markdown document

System Prompt (key excerpt):

```
You are an intelligence briefing writer. Produce a professional news briefing
in the exact Markdown structure below. Use a neutral, precise analytical tone.
Cite specific facts from the summaries — avoid generalities.

Required structure (use exactly these headings):
# [TOPIC] — News Briefing
**Date:** [today]
**Articles Reviewed:** [N]
**Overall Sentiment:** [sentiment]

## Executive Summary
## Key Themes
### [Theme name]
## Points of Tension
## Key Entities to Watch
## Sources Referenced
```

Why this prompt is shaped this way

Specifying the exact Markdown heading structure in the prompt ensures Step 6's critic receives a predictable document layout it can assess section by section. Without this constraint, the model varies its structure between runs, making the critique prompt far harder to write reliably. The "Cite specific facts" instruction directly counteracts the model's tendency toward generality when given synthesised input — by Step 5, the model has already seen a theme analysis and can default to restating the themes abstractly rather than grounding them in article-level facts.

Step 6 — Critic and Refiner

Call type	Two sequential LLM calls
Input (critic)	state["draft_briefing"] from Step 5
Output (critic)	state["critique"] — JSON with weaknesses, missing_angles, bias_flags
Input (refiner)	draft_briefing + critique JSON
Output (refiner)	state["final_briefing"] — polished Markdown with Editorial Notes

Critic System Prompt:

```
You are an editorial critic reviewing a news briefing.
Identify specific, actionable weaknesses only.

Respond ONLY with a valid JSON object — no markdown:
{
  "weaknesses"          : ["specific flaw 1", "specific flaw 2"],
  "missing_angles"      : ["angle not covered by the current draft"],
  "bias_flags"          : ["phrase or framing that appears biased"],
  "suggested_additions" : ["concrete thing that should be added"]
}
```

Refiner System Prompt:

```
You are a senior editor. Revise the news briefing using the critique below.
Keep the same Markdown structure. Address each weakness and missing angle
you can given the available data. Append this section at the end:

## Editorial Notes
[2-3 sentences: what was changed, what limitations remain]

Return the complete, revised briefing in Markdown — nothing else.
```

Why two sub-calls rather than one self-edit call

Testing a single "critique and revise yourself" prompt revealed a consistent failure mode: the model would produce a brief acknowledgment of its own weaknesses ("The briefing could benefit from more sources") followed by a revision that made only superficial changes. Separating critique from revision forces the model to commit to a specific, structured list of weaknesses before it writes the revision. The refiner then receives that list as an explicit task specification rather than a vague instruction to improve.

The "Identify specific, actionable weaknesses only" instruction was added after the critic began returning items like "the briefing is generally good" — a non-actionable observation that gave the refiner nothing to work with. Returning a JSON structure with named arrays (weaknesses, missing_angles, bias_flags) prevents this by making it structurally impossible to return praise rather than critique.